

# Architektury systemów komputerowych

## Lista zadań nr 9

Na zajęcia 6, 7 i 12 maja 2025

Kod należy prezentować przy pomocy [Compiler Explorer](#)<sup>1</sup> używając najnowszej dostępnej wersji kompilatora GCC.

**Zadanie 1.** Intencją procedury `swap` jest zamiana wartości przechowywanych w komórkach pamięci o adresie `xp` i `yp`. Odwołując się do pojęcia aliasingu pamięci (ang. *memory aliasing*) wytłumacz czemu kompilator nie może zoptymalizować poniższej procedury do procedury `swap2`? Pomóż mu zoptymalizować `swap` posługując się słowem kluczowym `restrict` i wyjaśnij jego znaczenie.

```
1 void swap(long *xp, long *yp) {
2     *xp = *xp + *yp; /* x+y */
3     *yp = *xp - *yp; /* x+y-y = x */
4     *xp = *xp - *yp; /* x+y-x = y */
5 }

1 void swap2(long *xp, long *yp) {
2     long x = *xp, y = *yp;
3     x = x + y, y = x - y, x = x - y;
4     *xp = x, *yp = y;
5 }
```

**Zadanie 2.** Ile razy zostanie zawołana funkcja `my_strlen` w funkcji `my_index` i dlaczego? Dodaj atrybut<sup>2</sup> `pure` do funkcji `my_strlen`. Czemu tym razem kompilator był w stanie lepiej zoptymalizować funkcję `my_index`? Czym charakteryzują się czyste funkcje? Następnie uzupełnij ciało funkcji `my_strlen` tak, by wykonywała to samo co `strlen`. Następnie usuń atrybut `pure` i dodając słowo kluczowe `static` zawęż zakres widoczności funkcji do bieżącej jednostki translacji. Co się stało w wyniku przeprowadzenia inliningu? Czy kompilatorowi udało się samemu wywnioskować, że funkcja jest czysta?

```
1 __attribute__((leaf))
2 size_t my_strlen(const char *s);
3
4 const char *my_index(const char *s, char v) {
5     for (size_t i = 0; i < my_strlen(s); i++)
6         if (s[i] == v)
7             return &s[i];
8     return 0;
9 }
```

**Zadanie 3.** Na wykładzie została zaprezentowana funkcja `compute` i na jej przykładzie wyjaśniono optymalizacje transformacji pętli, tj. zamiany pętli (ang. *loop interchange*), łączenia pętli (ang. *loop fusion*) oraz usuwania zmiennych indukcyjnych (ang. *induction variable elimination*). Czy kompilator był w stanie przeprowadzić te optymalizacje na poniższej funkcji? Jeśli nie, to zoptymalizuj ją krok po kroku. Przeanalizuj dokładnie działanie funkcji, czy jesteś w stanie poczynić jakieś obserwacje, które pozwalają Ci wykonać dodatkowe optymalizacje, np. *ponowne wyliczanie wartości* (ang. *rematerialization*<sup>3</sup>)?

```
1 void compute2(long *a, long *b, long k) {
2     long n = 1 << k;
3     for (long i = 0; i < n; i++)
4         a[i * n] = a[i] = 0;
5     for (long i = 1; i < n; i++)
6         for (long j = 1; j < n; j++)
7             a[j * n + i] = i * j;
8     for (long i = 1; i < n; i++)
9         for (long j = 1; j < n; j++)
10            b[i * n + j] = a[i * n + j]
11                + a[(i - 1) * n + (j - 1)];
12 }
```

<sup>2</sup><https://gcc.gnu.org/onlinedocs/gcc/Common-Function-Attributes.html>

<sup>3</sup><https://en.wikipedia.org/wiki/Rematerialization>

**Zadanie 4.** Na podstawie kodu wynikowego z kompilatora odtwórz zoptymalizowaną wersję funkcji foobar w języku C. Wskaż w poniższym kodzie niezmienniki pętli (ang. *loop invariants*), zmienne indukcyjne (ang. *induction variable*). Które wyrażenia zostały wyniesione przed pętlę i dlaczego? Które wyrażenia uległy osłabieniu (ang. *strength reduction*)?

```
1 void foobar(long a[], size_t n, long y, long z) {
2     for (int i = 0; i < n; i++) {
3         long x = y - z;
4         long j = 7 * i;
5         a[i] = j + x * x;
6     }
7 }
```

Poniższe zadania należy rozwiązywać na komputerze z systemem operacyjnym *Linux* dla platformy x86-64. Prowadzący zakłada, że zainstalowana dystrybucja będzie bazowała na *Debianie*. Do poniższej listy załączono na stronie przedmiotu pliki źródłowe wraz z plikiem *Makefile*.

**Zadanie 5.** Przeprowadź na swoim komputerze atak na program ropex wykorzystując podatność *przepętnienia bufora* w procedurze *echo*. Posłuż się techniką *ROP* (ang. *return oriented programming*). Wyznacz adresy gadżetów, tj. procedury gadget oraz dowolnej instrukcji *syscall* w pliku ropex (np. za pomocą narzędzia *objdump* z flagą *-d*). Wpisz je, w porządku *little-endian*, do pliku ropex.in.txt na pozycji 0x38 i 0x40, po czym przetłumacz go do postaci binarnej (za pomocą polecenia z pliku *Makefile*). Następnie uruchom polecenie *ropex ropex.in*, aby zobaczyć rezultat wykonania programu *nyancat*<sup>4</sup>. Przy pomocy *gdb* zaprezentuj zawartość stosu przed i po wykonaniu procedury *gets*. Pokaż, że procesor wykonując instrukcję *ret* skacze pod przygotowane przez Ciebie adresy.

**Wskazówka:** Wykaz poleceń i odnośnik do samouczka *gdb* podano na stronie przedmiotu w SKOS.

**Uwaga:** Jeżeli wynik kompilacji na Twoim komputerze będzie znacząco inny niż na komputerze prowadzącego, możesz być zmuszony do wykonania prostych modyfikacji, np. wpisania adresów na innych pozycjach. Możesz też zmodyfikować fragment ustawiający rejestr *y* w pliku *gadget.s*.

**Zadanie 6.** Zmodyfikuj opcje kompilacji programu ropex w pliku *Makefile*. Najpierw zleć kompilatorowi dodanie *kanarków* (ang. *canary*) włączając opcję *-fstack-protector*. Pokaż, że program wykrywa *uszkodzenie stosu* (ang. *stack smashing*). Posługując się debuggerem *gdb* zaprezentuj, że przy każdym uruchomieniu programu wartość kanarka jest inna. Następnie usuń opcję wymuszającą *statyczną konsolidację* *-static* i dodaj opcję *-fpie*, aby umożliwić *randomizację rozkładu przestrzeni adresowej* (ang. *Address Space Layout Randomization*). Pokaż, że adres *gadżetu* o nazwie *gadget* jest inny przy każdym uruchomieniu programu. Następnie dodaj opcję kompilacji *-z noexecstack*, która dodaje informacje o tym, że procesor nie powinien próbować wykonywać zawartości stosu. Przy pomocy programu *pmap* zweryfikuj, że istotnie stos uruchomionego programu ropex nie jest wykonywalny.

Czemu każde z zastosowanych wyżej zabezpieczeń utrudnia zadanie atakującemu?

**Uwaga!** Debugger *gdb* może wyłączyć ASLR przeprowadzaną przez konsolidator dynamiczny, aby ułatwić sobie pracę.

## Literatura

- [1] „*Computer Systems: A Programmer's Perspective*”  
Randal E. Bryant, David R. O'Hallaron; Pearson; 3rd edition, 2016
- [2] „*Modern Processor Design: Fundamentals of Superscalar Processors*”  
John Paul Shen, Mikko H. Lipasti; McGraw-Hill; 1st edition, 2005

---

<sup>4</sup>Do pliku ropex.in.txt można wpisać inny program pod warunkiem, że jego ścieżka będzie nie dłuższa.